

# Introduction

Given a set of data with target column included, we want to train a model that can learn to map the input features to the target. Machine Learning Models We are going to focus on 2 types of ML models which is RandomForest and XGB. Both of them are supervised in nature (labels are known and are to be predicted). We trained model on 4 wells and we use it to predict side pressure of the 5th well. We first write a function that will centralized our charts and also a code to silence any warning output that may hinder our model performing effectively after which we load in our dataset and proceed to data cleaning which is most important and first stage in every analysis. We the reported square root of the averaged squared difference between the target value and the value predicted by the model.

```
In [1]:  from IPython.core.display import HTML
        HTML("""
            <style>
            .output_png {
                display: table-cell;
                text-align: center;
                vertical-align: middle;
            }
            </style>
        """)
```

Out[1]:

```
In [2]:  import warnings
        from warnings import filterwarnings
        warnings.filterwarnings("ignore")
        filterwarnings("ignore", category=DeprecationWarning)
        filterwarnings("ignore", category=FutureWarning)
        filterwarnings("ignore", category=UserWarning)
```

```
In [3]:  import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import seaborn as sns
```

```
In [4]:  pd.set_option('display.max_columns', None)
```

```
In [5]:  df = pd.read_excel("Aadatset.xlsx")
```

```
In [6]: df.head()
```

```
Out[6]:
```

|   | Well_Name | Well_Stage | Time | Discharge_Rate_ADC1 | prop_rate | Perc_Well | Perc_Stage |
|---|-----------|------------|------|---------------------|-----------|-----------|------------|
| 0 | W1        | 1          | 7    | 28.469387           | 0.000000  | 0.032258  | 0.001690   |
| 1 | W1        | 1          | 107  | 34.991882           | 0.123095  | 0.032258  | 0.025827   |
| 2 | W1        | 1          | 207  | 41.274349           | 0.048407  | 0.032258  | 0.049964   |
| 3 | W1        | 1          | 307  | 43.956402           | 13.609436 | 0.032258  | 0.074101   |
| 4 | W1        | 1          | 407  | 44.947819           | 0.000000  | 0.032258  | 0.098238   |

## Data Cleaning and Analysis

- Load the data using pandas and then checked for a constant variable, two of all the variables are constant so they were dropped/removed
- Null values is checked for
- The distribution of the target variable was checked
- The target variable was also plotted using line graph
- The correlation between variables was plotted using a heatmap
- Bosplot was used to determine if some the variables have outlier

```
In [7]: # check for column that is not constant
not_const = [i for i in df.columns.tolist() if len(df[i].unique().tolist())
```

```
In [8]: #check for column that is constant
const = [i for i in df.columns if i not in not_const]
const
```

```
Out[8]: ['perf_SZHOLENOM', 'casing_SZODNOM']
```

## Missing value

Checking for missing value, it is important to check for missing value in our data. Missing data or missing values, occur when no data value is stored for the variable in an observation. Missing data are a common occurrence, however data can go missing due to incomplete data entry, equipment malfunctions, lost files, and many other reasons. But here in our data there is no account of missing value exit in all our variables.

```
In [9]:  # Checking for missing value
df.isna().sum()
```

```
Out[9]: Well_Name      0
Well_Stage      0
Time            0
Discharge_Rate_ADC1  0
prop_rate       0
Perc_Well       0
Perc_Stage      0
inline_prop_conc  0
UB_Isopach      0
MB_Isopach      0
LB_Isopach      0
Sanish_Isopach  0
Basal_Bakken_Isopach  0
TF1_reservoir_Isopach  0
TF1_mudstone_isopach  0
TF2_reservoir_isopach  0
TF2_mudstone_isopach  0
LTF_Isopach     0
Geo_Pressure    0
Geo_Frac_Closure_Pressure  0
Geo_Perm        0
Geo_KH          0
WV_clusters_per_stage  0
perf_max_depth_MD  0
perf_shot_bottom_depth_MD  0
perf_shot_top_depth_MD  0
perf_phasing    0
perf_shot_density  0
perf_shot_plan  0
perf_SZHOLEACT  0
perf_SZHOLENOM  0
casing_DEPTHBTMCALC  0
casing_DEPHTOPCALC  0
casing_SZODNOM   0
Wellside_Pressure  0
dtype: int64
```

```
In [10]:  ## remove the constant variable from the data 'perf_SZHOLENOM', 'casing_SZ
df.drop(const, axis=1, inplace=True)
```

We check for a unique value for the categorical variable, we have 5 unique variables w1, w2, w3, w4 and w5. We convert the categorical data to numerical data because machine learning model will not recognize if it is not converted to a numerical variable, therefore, w1 is categorized as 1, w2 categorized as 2, w3 categorized as 3, w4 categorized as 4 and w5 categorized as 5 for our model to run effectively.

```
In [11]:  df["Well_Name"].unique()
```

```
Out[11]: array(['W1', 'W2', 'W3', 'W4', 'W5'], dtype=object)
```

```
In [12]: # transform a categorical variable to a numeral variable
df["Well_Name"] = df["Well_Name"].map({'W1':1, 'W2':2, 'W3':3, 'W4':4, 'W5':5})

In [13]: #we preview the first five rows of our data after the categorical variable
df.head()
```

```
Out[13]:
```

|   | Well_Name | Well_Stage | Time | Discharge_Rate_ADC1 | prop_rate | Perc_Well | Perc_Stage |
|---|-----------|------------|------|---------------------|-----------|-----------|------------|
| 0 | 1         | 1          | 7    | 28.469387           | 0.000000  | 0.032258  | 0.001690   |
| 1 | 1         | 1          | 107  | 34.991882           | 0.123095  | 0.032258  | 0.025827   |
| 2 | 1         | 1          | 207  | 41.274349           | 0.048407  | 0.032258  | 0.049964   |
| 3 | 1         | 1          | 307  | 43.956402           | 13.609436 | 0.032258  | 0.074101   |
| 4 | 1         | 1          | 407  | 44.947819           | 0.000000  | 0.032258  | 0.098238   |

## Descriptive Statistic

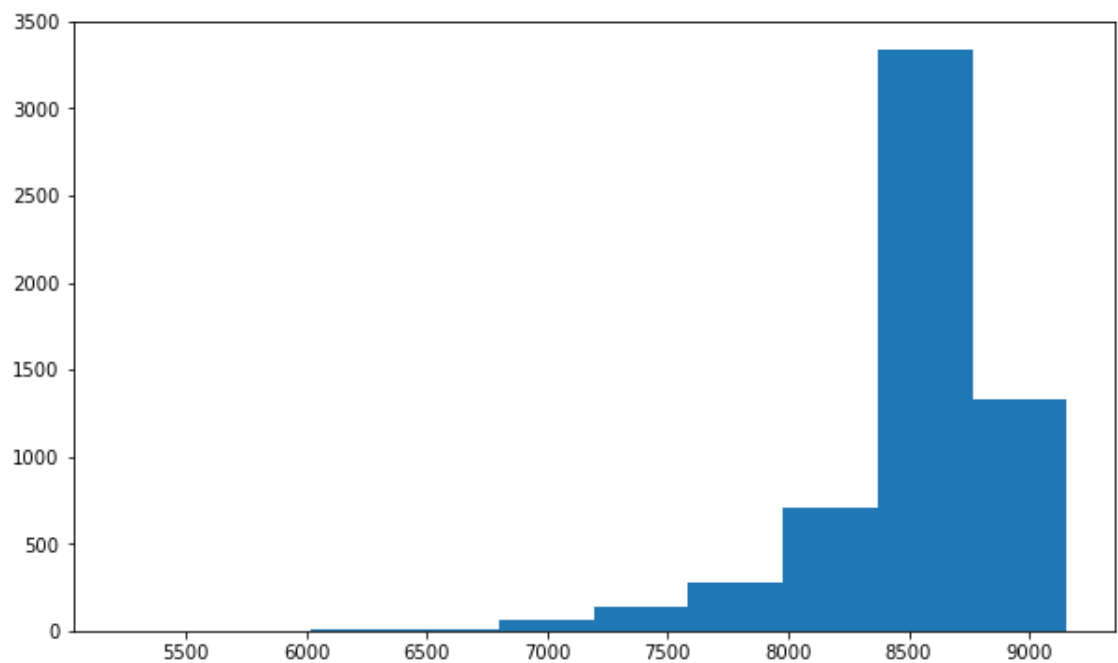
Basic statistics of all the variables We show the basic statistics for our data set,, such as mean, standard deviation, minimum value, maximum value and so on for all the variables in our data set. We plot a histogram plot which is a popular graphing tool. It is used to summarize discrete or continuous data that are measured on an interval scale. It is often used to illustrate the major features of the distribution of the data in a convenient form also we run a plot for "Wellside\_Pressure which shown the pattern of Wellside\_Pressure variable. Correlation plot correlation ranges from -1 to 1. It gives us an indication on two things: The direction of the relationship between the 2 variables and the strength of the relationship between the 2 variables regarding the direction of the relationship: On the one hand, a negative correlation implies that the two variables under consideration vary in opposite directions, that is, if a variable increases the other decreases and vice versa. On the other hand, a positive correlation implies that the two variables under consideration vary in the same direction, i.e., if a variable increases the other one increases and if one decreases the other one decreases as well. Regarding the strength of the relationship: The more extreme the correlation coefficient (the closer to -1 or 1), the stronger the relationship. This also means that a correlation close to 0 indicates that the two variables are independent, that is, as one variable increases, there is no tendency in the other variable to either decrease or increase. Below is the correlation plot.

```
In [14]: ▶ # describe funtion takes all numeric columns and provide descriptive stati
# descriptive statistics include - count of rows, mean, standard deviation
# Note: Quartile 50 = median
df.describe()
```

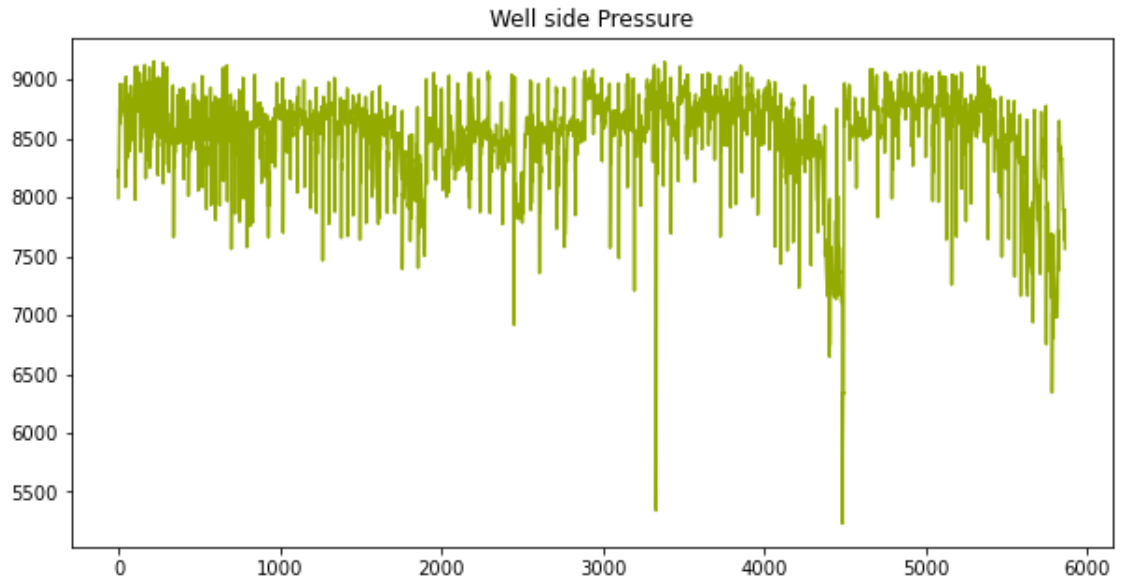
```
Out[14]:
```

|              | Well_Name   | Well_Stage  | Time        | Discharge_Rate_ADC1 | prop_rate   | Perc_\   |
|--------------|-------------|-------------|-------------|---------------------|-------------|----------|
| <b>count</b> | 5863.000000 | 5863.000000 | 5863.000000 | 5863.000000         | 5863.000000 | 5863.000 |
| <b>mean</b>  | 3.229405    | 17.901927   | 2178.799761 | 61.770225           | 67.975136   | 0.539    |
| <b>std</b>   | 1.365389    | 9.621306    | 1444.346607 | 14.006056           | 49.924066   | 0.284    |
| <b>min</b>   | 1.000000    | 1.000000    | 1.000000    | 13.806818           | 0.000000    | 0.027    |
| <b>25%</b>   | 2.000000    | 10.000000   | 1011.500000 | 51.793831           | 25.549805   | 0.297    |
| <b>50%</b>   | 3.000000    | 18.000000   | 2023.000000 | 59.975651           | 67.820312   | 0.540    |
| <b>75%</b>   | 4.000000    | 26.000000   | 3120.500000 | 70.562965           | 107.480469  | 0.800    |
| <b>max</b>   | 5.000000    | 37.000000   | 8310.000000 | 90.229591           | 163.046875  | 1.000    |

```
In [15]: ▶ #histogram plot
plt.figure(figsize=(10, 6))
_ = plt.hist(df["Wellside_Pressure"])
```



```
In [16]: ▶ _ = df["Wellside_Pressure"].plot(figsize=(10, 5), title='Well side Pressur
```



## Heatmap Correlation

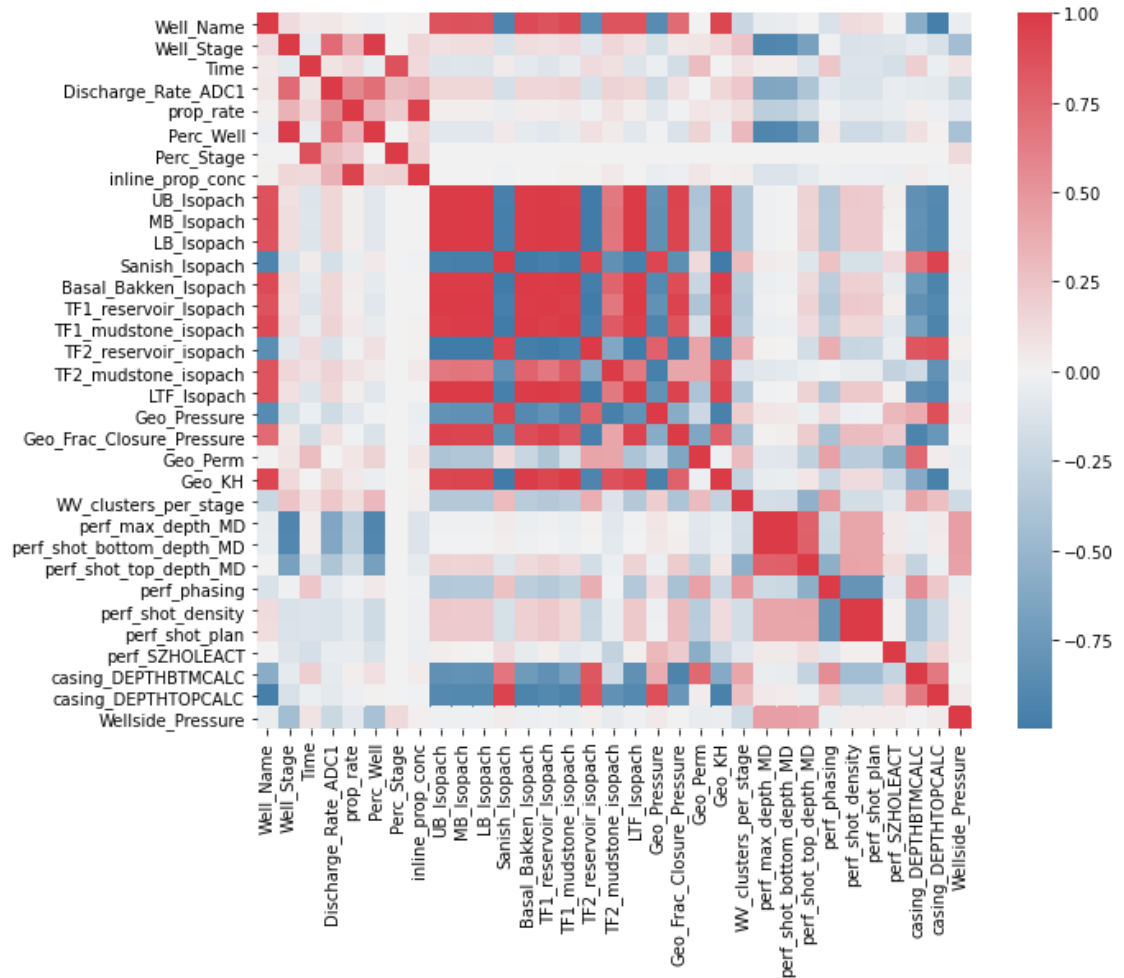
Correlation Matrix with Heatmap Correlation states how the features are related to each other or the target variable.

Correlation can be positive (increase in one value of feature increases the value of the target variable) or negative (increase in one value of feature decreases the value of the target variable)

Heatmap makes it easy to identify which features are most related to the target variable, we will plot heatmap of correlated features using the seaborn library.

```
In [17]: #correlation plot
f, ax = plt.subplots(figsize=(10, 8))
corr = df.corr()
sns.heatmap(corr, mask=np.zeros_like(corr, dtype=np.bool), cmap=sns.diverg
```

Out[17]: <AxesSubplot:>



```
In [18]: ▶ sm = [i for i in df.columns.tolist() if df[i].mean() < 100]
sm
```

```
Out[18]: ['Well_Name',
          'Well_Stage',
          'Discharge_Rate_ADC1',
          'prop_rate',
          'Perc_Well',
          'Perc_Stage',
          'inline_prop_conc',
          'UB_Isopach',
          'MB_Isopach',
          'LB_Isopach',
          'Sanish_Isopach',
          'Basal_Bakken_Isopach',
          'TF1_reservoir_Isopach',
          'TF1_mudstone_isopach',
          'TF2_reservoir_isopach',
          'TF2_mudstone_isopach',
          'WV_clusters_per_stage',
          'perf_shot_density',
          'perf_shot_plan',
          'perf_SZHOLEACT']
```

## Box Plot

In descriptive statistics, a boxplot sometime called whisker plot is a type of chart adequately used in explanatory data analysis. Box plots visually show the distribution of numerical data and skewness through displaying the data quartiles (or percentiles) and averages.

Box plots show the five-number summary of a set of data: including the minimum score, first (lower) quartile, median, third (upper) quartile, and maximum score. However box plots are helps as they provide a visual summary of the data enabling researchers to quickly identify mean values, the dispersion of the data set, and signs of skewness.

The median (middle quartile) marks the mid-point of the data and is shown by the line that divides the box into two parts. Half the scores are greater than or equal to this value and half are less. Inter-quartile range: The middle “box” represents the middle 50% of scores for the group. The range of scores from lower to upper quartile is referred to as the inter-quartile range. The middle 50% of scores fall within the inter-quartile range. Upper quartile: Seventy-five percent of the scores fall below the upper quartile. Lower quartile: Twenty-five percent of scores fall below the lower quartile. Whiskers: The upper and lower whiskers represent scores outside the middle 50%. Whiskers often (but not always) stretch over a wider range of scores than the middle quartile groups.

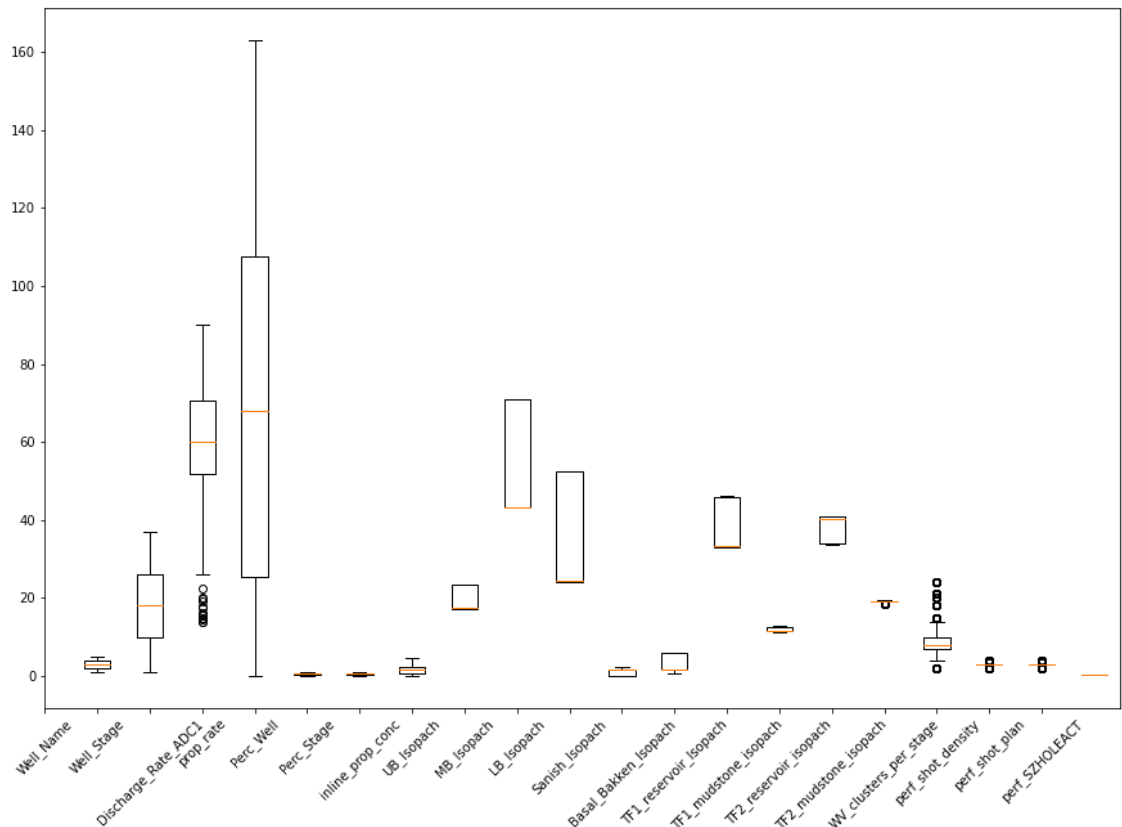
In general, when the median is in the middle of the box, and the whiskers are about the same on both sides of the box, then the distribution is symmetric. When the median is closer to the bottom of the box, and if the whisker is shorter on the lower end of the box, then the distribution is positively skewed (skewed right).

When the median is closer to the top of the box, and if the whisker is shorter on the upper end of the box, then the distribution is negatively skewed (skewed left).



The box plot of well stage is comparatively short, this suggests that overall responses have a high level of agreement with each other. The box plot of perc\_well is comparatively tall which suggests responses hold quite different opinions about the variable

```
In [19]: ▶ #boxplot
plt.figure(figsize=(15, 10))
s = df[sm]
_ = plt.boxplot(s)
_ = plt.xticks(ticks=np.arange(len(s.columns)), labels=s.columns.tolist(),
```

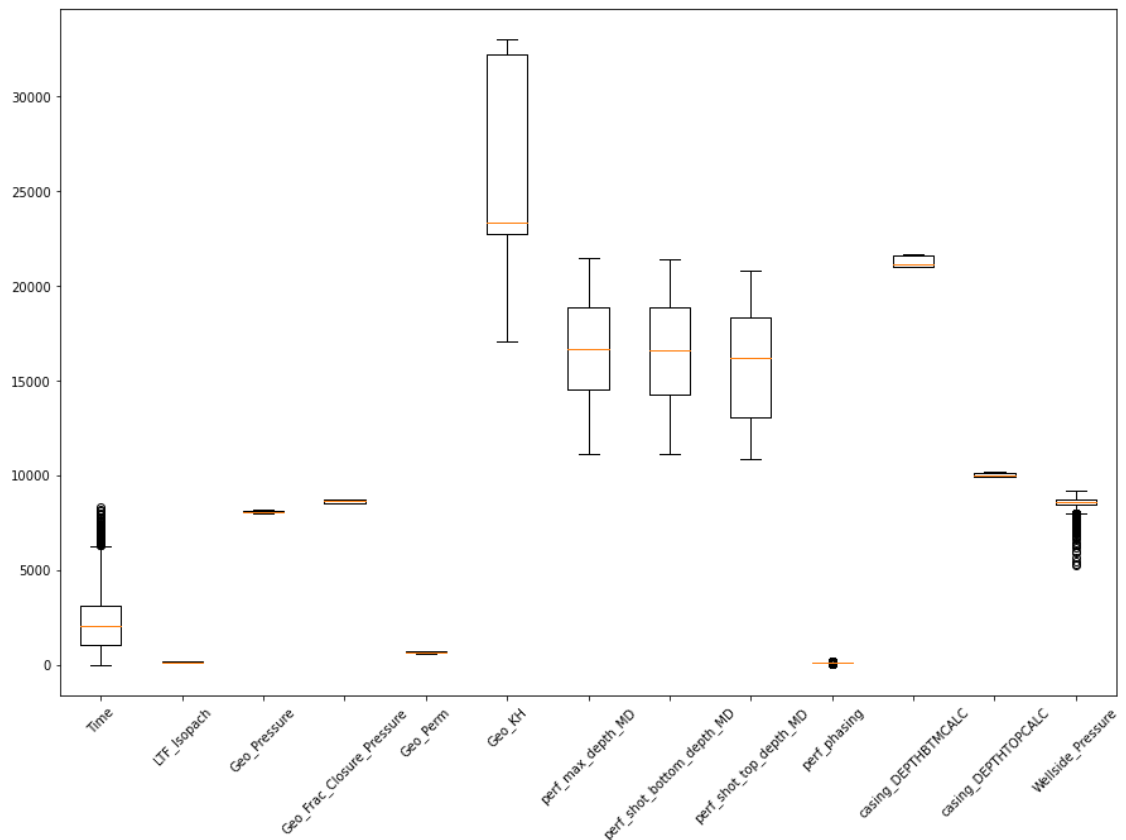


```
In [20]: ▶ # the below code will print the number of rows and columns
l = df[[i for i in df.columns if i not in sm]]
l.shape
```

Out[20]: (5863, 13)

```
In [21]: #boxplot
plt.figure(figsize=(15, 10))

_ = plt.boxplot(l)
_ = plt.xticks(ticks=np.arange(1, len(l.columns)+1, 1), labels=l.columns.t
```



```
In [ ]:
```

## Data grouping

Before we group our data into training and testing, there is need to check and confirm our data information. The below code will display the column names along with their data type int64 = integer numbers float = floating point numbers object = text or mixed numeric or non-numeric values

Note: Machine learning models require all input and output variables to be numeric. Run the model with data as-is and then iterate for feature engineering. This means that if your data contains categorical data, you must encode it to numbers before you can fit and evaluate a model which we have done in case of "Well\_Name" variable.

Ordinal encoding - The integer values have a natural ordered relationship between each other and machine learning algorithms may be able to understand and harness this relationship.

```
In [22]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5863 entries, 0 to 5862
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Well_Name                            5863 non-null   int64
1   Well_Stage                           5863 non-null   int64
2   Time                                 5863 non-null   int64
3   Discharge_Rate_ADC1                 5863 non-null   float64
4   prop_rate                           5863 non-null   float64
5   Perc_Well                           5863 non-null   float64
6   Perc_Stage                           5863 non-null   float64
7   inline_prop_conc                     5863 non-null   float64
8   UB_Isopach                           5863 non-null   float64
9   MB_Isopach                           5863 non-null   float64
10  LB_Isopach                           5863 non-null   float64
11  Sanish_Isopach                       5863 non-null   float64
12  Basal_Bakken_Isopach                 5863 non-null   float64
13  TF1_reservoir_Isopach                5863 non-null   float64
14  TF1_mudstone_isopach                 5863 non-null   float64
15  TF2_reservoir_isopach                5863 non-null   float64
16  TF2_mudstone_isopach                 5863 non-null   float64
17  LTF_Isopach                          5863 non-null   float64
18  Geo_Pressure                         5863 non-null   float64
19  Geo_Frac_Closure_Pressure            5863 non-null   float64
20  Geo_Perm                             5863 non-null   float64
21  Geo_KH                              5863 non-null   float64
22  WV_clusters_per_stage                5863 non-null   float64
23  perf_max_depth_MD                   5863 non-null   float64
24  perf_shot_bottom_depth_MD            5863 non-null   float64
25  perf_shot_top_depth_MD               5863 non-null   float64
26  perf_phasing                         5863 non-null   float64
27  perf_shot_density                    5863 non-null   float64
28  perf_shot_plan                       5863 non-null   float64
29  perf_SZHOLEACT                       5863 non-null   float64
30  casing_DEPTHBTM CALC                 5863 non-null   float64
31  casing_DEPTHHTOP CALC                5863 non-null   float64
32  Wellside_Pressure                    5863 non-null   float64
dtypes: float64(30), int64(3)
memory usage: 1.5 MB
```

## Splitting the data

Split into train and validation set Train – what we use to train the model

Validation – what we use to evaluate the model

Test – data that is unexposed to the model X\_train: independent feature data for training the model y\_train: dependent feature data for training the model X\_test: independent feature data for testing the model; will be used to predict the target values y\_test: original target values of X\_test; We will compare this values with our predicted values.

test\_size = 5th wall of the data will go for test set and 1st to 4th wall of the data will go for train set

```
In [23]: ▶ # separate train and test data using the 5Th wall
```

```
train = df[df["Well_Name"] != 5]
test = df[df["Well_Name"] == 5]
```

```
In [27]: ▶ # show the size of the train and test data
train.shape, test.shape
```

```
Out[27]: ((4492, 33), (1371, 33))
```

```
In [28]: ▶ # divide the data into dependent and independend variables
```

```
X = df.drop("Wellside_Pressure", 1)
y = df["Wellside_Pressure"]
```

```
In [29]: ▶ from sklearn.model_selection import train_test_split
```

```
# divide the train data to train and validation
n_train = 4492
X_train, X_test = train.drop("Wellside_Pressure", 1), test.drop("Wellside_Pressure", 1)
y_train, y_test = train["Wellside_Pressure"], test["Wellside_Pressure"]
X_train.shape, y_train.shape
```

```
Out[29]: ((4492, 32), (4492,))
```

## Model Selection Approach

Choosing from three different models by training and testing using the train and validation data, the three models considering is Random Forest Regressor, Xtreme Gradient Boosting Regressor and Categorical Boosting Regressor

There are three primary metrics used to evaluate models. These are: Mean absolute error (MAE), Mean squared error (MSE), or Root mean squared error (RMSE). Mean Squared Error: It is simply the average of the squared difference between the target value and the value predicted by the regression model. Root Mean Squared Error: It is the square root of the averaged squared difference between the target value and the value predicted by the model and Mean Absolute Error: It is the absolute difference between the target value and the value predicted by the model.

We reported the root mean square error (RMSE) of the model.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision.

```
In [33]: > #imports
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from xgboost import plot_importance
from catboost import CatBoostRegressor
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import MinMaxScaler
```

```
In [34]: > rmse_t = []
```

```
In [35]: > # modify the mean square error to Root mean square error
def rmse(true, pred):
    return np.sqrt(mean_squared_error(true, pred))
```

```
In [36]: > rf = RandomForestRegressor()
rf.fit(X_train, y_train)
```

```
Out[36]: RandomForestRegressor()
```

```
In [37]: > #rf.score(X_train, y_train)
rmse(rf.predict(X_train), y_train)
#rmse_tr.append(rmse(rf.predict(X_train), y_train))
```

```
Out[37]: 50.273880418640395
```

```
In [38]: > rf_pred = rf.predict(X_test)
rmse(rf_pred, y_test)
```

```
Out[38]: 299.9494987014369
```

```
In [39]: > rmse_t.append(rmse(rf.predict(X_test), y_test))
```

```
In [40]: > xg = XGBRegressor()
xg.fit(X_train, y_train)
```

```
Out[40]: XGBRegressor(base_score=0.5, booster='gbtree', colsample_bylevel=1,
    colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
    importance_type='gain', interaction_constraints='',
    learning_rate=0.300000012, max_delta_step=0, max_depth=6,
    min_child_weight=1, missing=nan, monotone_constraints='()',
    n_estimators=100, n_jobs=0, num_parallel_tree=1, random_stat
e=0,
    reg_alpha=0, reg_lambda=1, scale_pos_weight=1, subsample=1,
    tree_method='exact', validate_parameters=1, verbosity=None)
```

```
In [41]: > #rf.score(X_train, y_train)
rmse(xg.predict(X_train), y_train)
#rmse_tr.append(rmse(xg.predict(X_train), y_train))
```

```
Out[41]: 42.743357439200196
```

```
In [42]:  xg_pred = xg.predict(X_test)
          rmse(xg_pred, y_test)
```

Out[42]: 290.4742044434364

```
In [43]:  rmse_t.append(rmse(xg.predict(X_test), y_test))
```

```
In [44]:  cat = CatBoostRegressor()
          cat.fit(X_train, y_train, verbose=False)
```

Out[44]: <catboost.core.CatBoostRegressor at 0x25a1b99aa90>

```
In [45]:  #rf.score(X_train, y_train)
          rmse(cat.predict(X_train), y_train)
          #rmse_tr.append(rmse(cat.predict(X_train), y_train))
```

Out[45]: 73.95577725468686

```
In [ ]:  
```

```
In [46]:  cat_pred = cat.predict(X_test)
          rmse(cat_pred, y_test)
```

Out[46]: 287.02662766992125

```
In [47]:  rmse_t.append(rmse(cat_pred, y_test))
```

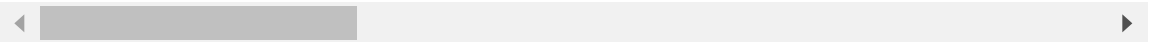
```
In [48]:  # normalize features
          scaler = MinMaxScaler(feature_range=(0, 1))
          scaled = scaler.fit_transform(df)
```

In [49]: `pd.DataFrame(scaler.inverse_transform(scaled))`

Out[49]:

|      | 0   | 1    | 2      | 3         | 4         | 5        | 6        | 7        | 8         |      |
|------|-----|------|--------|-----------|-----------|----------|----------|----------|-----------|------|
| 0    | 1.0 | 1.0  | 7.0    | 28.469387 | 0.000000  | 0.032258 | 0.001690 | 0.000000 | 17.482805 | 43.3 |
| 1    | 1.0 | 1.0  | 107.0  | 34.991882 | 0.123095  | 0.032258 | 0.025827 | 0.005025 | 17.482805 | 43.3 |
| 2    | 1.0 | 1.0  | 207.0  | 41.274349 | 0.048407  | 0.032258 | 0.049964 | 0.001675 | 17.482805 | 43.3 |
| 3    | 1.0 | 1.0  | 307.0  | 43.956402 | 13.609436 | 0.032258 | 0.074101 | 0.442289 | 17.482805 | 43.3 |
| 4    | 1.0 | 1.0  | 407.0  | 44.947819 | 0.000000  | 0.032258 | 0.098238 | 0.000000 | 17.482805 | 43.3 |
| ...  | ... | ...  | ...    | ...       | ...       | ...      | ...      | ...      | ...       | ...  |
| 5858 | 5.0 | 37.0 | 3804.0 | 87.004868 | 35.835938 | 1.000000 | 0.888993 | 0.588396 | 23.393555 | 71.0 |
| 5859 | 5.0 | 37.0 | 3904.0 | 83.091370 | 0.000000  | 1.000000 | 0.912363 | 0.000000 | 23.393555 | 71.0 |
| 5860 | 5.0 | 37.0 | 4004.0 | 83.383583 | 0.000000  | 1.000000 | 0.935733 | 0.000000 | 23.393555 | 71.0 |
| 5861 | 5.0 | 37.0 | 4104.0 | 82.068649 | 0.000000  | 1.000000 | 0.959103 | 0.000000 | 23.393555 | 71.0 |
| 5862 | 5.0 | 37.0 | 4204.0 | 77.508118 | 0.000000  | 1.000000 | 0.982473 | 0.000000 | 23.393555 | 71.0 |

5863 rows × 33 columns



```
In [50]: aa = ["Random Forest", "Xgboost", "Catboost"]
plt.figure(figsize=(8, 6))
plt.plot(aa, rmse_t, marker='.', label="score")
plt.ylabel('RMSE Score', size=15)
plt.xlabel('Models', size=15)
plt.title("Root Mean Squire Error Scores")
plt.legend(fontsize=15)
plt.show()
```



Out of the three selected model Categorical Boosting performs better than the rest so CatBoost will be used as the final mode

## Feature selection

- **Feature selection** or **variable selection** is the process of selecting a subset of relevant features or variables from the total features of a level in a data set to build machine learning algorithms. Feature Selection Feature Selection is one of the core concepts in machine learning which hugely impacts the performance of your model. The data features that one use to train your machine learning models have a huge influence on the performance one can achieve.

Irrelevant or partially relevant features can negatively impact model performance.

Benefits of performing feature selection: Reduces Overfitting: Less redundant data means less opportunity to make decisions based on noise(irrelevant data). Improves Performance: Less misleading data means model's performance improves. Reduces Training Time: Less data



means that algorithms train faster. Recursive Feature Elimination (RFE) Feature Importance using Random Forest Boruta XGBoost All the techniques will be implemented independent of each other.

## Advantages of selecting features

- There are various advantages of feature selection process. These are as follows:-

1. Improved accuracy
2. Simple models are easier to interpret.
3. Shorter training times
4. Enhanced generalization by reducing Overfitting
5. Easier to implement by software developers
6. Reduced risk of data errors by model use

```
In [51]: ▶ # using sklearn variancethreshold to find constant features
from sklearn.feature_selection import SelectFromModel
from sklearn.feature_selection import VarianceThreshold
sel = VarianceThreshold(threshold=0)
sel.fit(X_train) # fit finds the features with zero variance
```

```
Out[51]: VarianceThreshold(threshold=0)
```

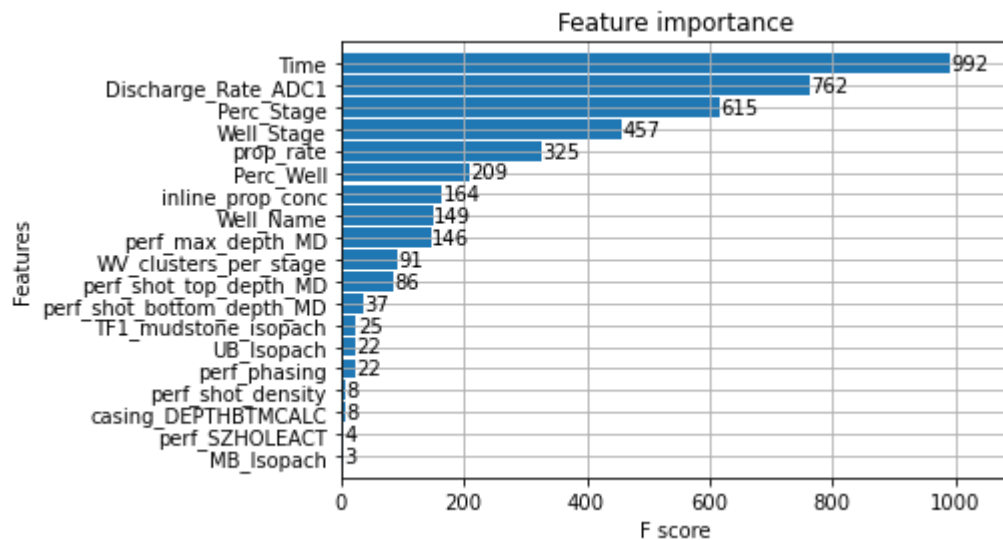
```
In [52]: ▶ # get_support is a boolean vector that indicates which features are retained
# if we sum over get_support, we get the number of features that are not constant
sum(sel.get_support())
```

```
Out[52]: 32
```

```
In [54]: ▶ # a = xg.feature_importances_
# col = np.where(a > 0, 1, 0)
# col
```

```
In [55]: ▶ #[i for i in col X.columns]
```

```
In [56]: _ = plot_importance(xg, height=0.9)
```



```
In [57]: col = []
```

## Description of Algorithm

### What is CatBoost

**CatBoost** is the leading model for working with standard tabular data (the type of data you store in Pandas DataFrames, as opposed to more exotic types of data like images and videos). CatBoost models dominate many Kaggle competitions.

CatBoost is an implementation of the **Gradient Boosted Decision Trees** algorithm (scikit-learn has another version of this algorithm, but XGBoost has some technical advantages.) What is **Gradient Boosted Decision Trees**? We'll walk through a diagram.

 CatBoost image

We go through cycles that repeatedly builds new models and combines them into an **ensemble** model. We start the cycle by calculating the errors for each observation in the dataset. We then build a new model to predict those. We add predictions from this error-predicting model to the "ensemble of models."

To make a prediction, we add the predictions from all previous models. We can use these predictions to calculate new errors, build the next model, and add it to the ensemble.

There's one piece outside that cycle. We need some base prediction to start the cycle. In practice, the initial predictions can be pretty naive. Even if it's predictions are wildly inaccurate, subsequent additions to the ensemble will address those errors.

This process may sound complicated, but the code to use it is straightforward. We'll fill in some additional explanatory details in the **model tuning** section below.

## Hyperparameter Tuning

Hyperparameter Optimization Steps to tune the parameters:

Prioritize those parameters which have the most effect on our model. Set various values to these parameters and store them in a dictionary as shown below. Create an object of the GridSearchCV class and assign the parameters to it. Fit the training set in the object. We will get the best parameters from the bestestimator property of the object. Use this object to fit training set to your classifier.

In [58]: `!pip install optuna`

```
Requirement already satisfied: optuna in c:\users\someone\anaconda\anaconda\lib\site-packages (3.0.2)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: Ignoring invalid distribution - (c:\users\someone\anaconda\anaconda\lib\site-packages)
```

```
WARNING: You are using pip version 22.1.2; however, version 22.2.2 is available.
```

```
You should consider upgrading via the 'C:\Users\Someone\anaconda\Anaconda\python.exe -m pip install --upgrade pip' command.
```

Requirement already satisfied: alembic>=1.5.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (1.8.1)

Requirement already satisfied: scipy<1.9.0,>=1.7.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (1.7.3)

Requirement already satisfied: packaging>=20.0 in c:\users\someone\appdata\roaming\python\python38\site-packages (from optuna) (21.3)

Requirement already satisfied: cliff in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (4.0.0)

Requirement already satisfied: sqlalchemy>=1.3.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (1.4.32)

Requirement already satisfied: cmaes>=0.8.2 in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (0.8.2)

Requirement already satisfied: numpy in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (1.22.4)

Requirement already satisfied: PyYAML in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (6.0)

Requirement already satisfied: tqdm in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (4.56.2)

Requirement already satisfied: colorlog in c:\users\someone\anaconda\anaconda\lib\site-packages (from optuna) (6.7.0)

Requirement already satisfied: importlib-resources in c:\users\someone\anaconda\anaconda\lib\site-packages (from alembic>=1.5.0->optuna) (5.9.0)

Requirement already satisfied: Mako in c:\users\someone\anaconda\anaconda\lib\site-packages (from alembic>=1.5.0->optuna) (1.2.3)

Requirement already satisfied: importlib-metadata in c:\users\someone\anaconda\anaconda\lib\site-packages (from alembic>=1.5.0->optuna) (4.11.3)

Requirement already satisfied: pyparsing!=3.0.5,>=2.0.2 in c:\users\someone\anaconda\anaconda\lib\site-packages (from packaging>=20.0->optuna) (3.0.4)

Requirement already satisfied: greenlet!=0.4.17 in c:\users\someone\anaconda\anaconda\lib\site-packages (from sqlalchemy>=1.3.0->optuna) (1.1.1)

Requirement already satisfied: PrettyTable>=0.7.2 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cliff->optuna) (3.4.1)

Requirement already satisfied: stevedore>=2.0.1 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cliff->optuna) (4.0.0)

Requirement already satisfied: cmd2>=1.0.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cliff->optuna) (2.4.2)

Requirement already satisfied: autopage>=0.4.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cliff->optuna) (0.5.1)

Requirement already satisfied: wcwidth>=0.1.7 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cmd2>=1.0.0->cliff->optuna) (0.2.5)

Requirement already satisfied: pyperclip>=1.6 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cmd2>=1.0.0->cliff->optuna) (1.8.2)

Requirement already satisfied: attrs>=16.3.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cmd2>=1.0.0->cliff->optuna) (21.4.0)

Requirement already satisfied: pyreadline3 in c:\users\someone\anaconda\anaconda\lib\site-packages (from cmd2>=1.0.0->cliff->optuna) (3.4.1)

Requirement already satisfied: zipp>=0.5 in c:\users\someone\anaconda\anaconda\lib\site-packages (from importlib-metadata->alembic>=1.5.0->optuna) (3.8.0)

Requirement already satisfied: pbr!=2.1.0,>=2.0.0 in c:\users\someone\anaconda\anaconda\lib\site-packages (from stevedore>=2.0.1->cliff->optuna) (5.10.0)

Requirement already satisfied: colorama in c:\users\someone\anaconda\anaconda\lib\site-packages (from colorlog->optuna) (0.4.4)

Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\someone\anaconda\anaconda\lib\site-packages (from colorlog->optuna) (2.0.1)

```
onda\anaconda\lib\site-packages (from Mako->alembic>=1.5.0->optuna) (2.0.1)
```

```
In [59]: ➤ from hyperopt import STATUS_OK, Trials, fmin, hp, tpe
import optuna
```

```
In [60]: ➤ def check_rmse(model, x_val, y_val):
    pred = model.predict(x_val)
    return np.sqrt(mean_squared_error(y_val, pred))
```

```
In [61]: ➤ def objective(trial):
    param = {}
    param['learning_rate'] = trial.suggest_discrete_uniform("learning_rate", 0.001, 0.1, 0.01)
    param['depth'] = trial.suggest_int('depth', 5, 11)
    param['l2_leaf_reg'] = trial.suggest_discrete_uniform('l2_leaf_reg', 1, 10, 1)
    param['min_child_samples'] = trial.suggest_categorical('min_child_samples', [5, 10, 20])
    param['grow_policy'] = 'Depthwise'
    param['iterations'] = 8000
    param['use_best_model'] = True
    param['eval_metric'] = 'RMSE'
    param['od_type'] = 'iter'
    param['od_wait'] = 50
    param['random_state'] = 42
    param['logging_level'] = 'Silent'

    regressor = CatBoostRegressor(**param)
    regressor.fit(X_train.copy(), y_train.copy(),
                  eval_set=[(X_test.copy(), y_test.copy())],
                  early_stopping_rounds=100)
    return check_rmse(regressor, X_test, y_test)
```

```
In [124]: ➤ %%time
study = optuna.create_study(study_name='catboost-seed')
study.optimize(objective, n_trials=100, n_jobs=-1, timeout=24000)
```

```
In [70]: ➤ study.best_value
```

```
Out[70]: 278.9695091727414
```

```
In [71]: ➤ print("The best hyperparameters are : ", "\n")
study.best_params
```

The best hyperparameters are :

```
Out[71]: {'learning_rate': 0.009,
          'depth': 6,
          'l2_leaf_reg': 1.0,
          'min_child_samples': 8}
```

```
In [ ]: ➤
```

```
In [62]: ▶ cat = CatBoostRegressor(learning_rate= 0.009, depth= 6, l2_leaf_reg= 1.0,  
cat.fit(X_train, y_train, verbose=False)
```

```
Out[62]: <catboost.core.CatBoostRegressor at 0x25a1ed8fb80>
```

```
In [63]: ▶ cat.score(X_train, y_train)
```

```
Out[63]: 0.8451197400862396
```

```
In [64]: ▶ cat.score(X_test, y_test)
```

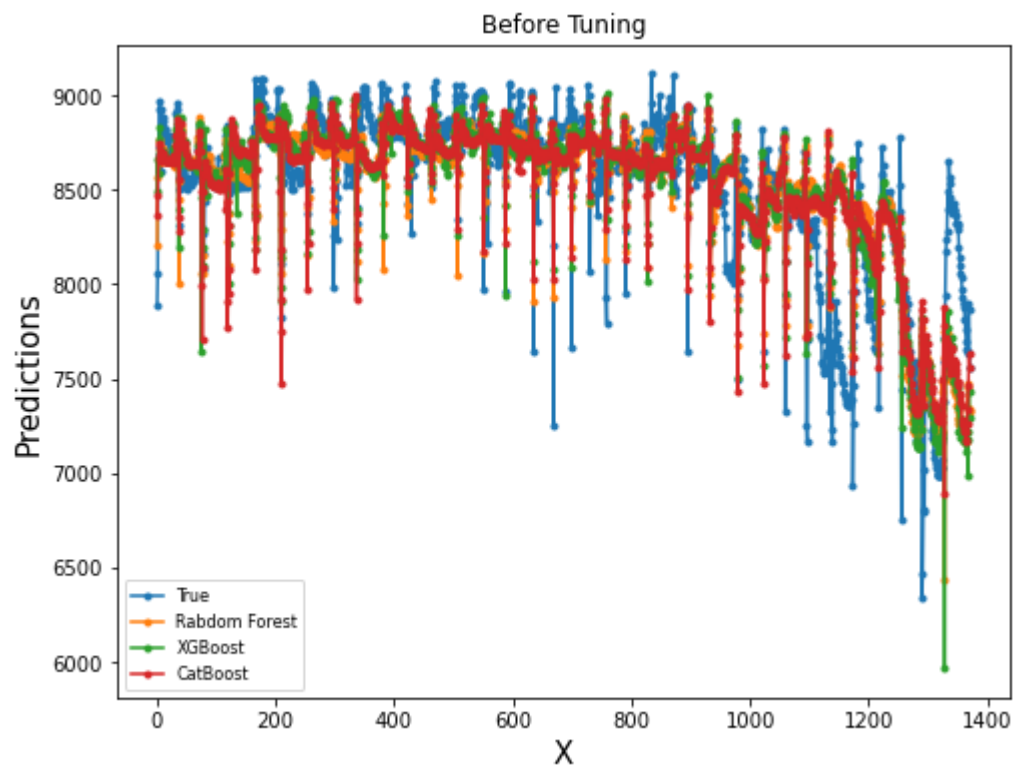
```
Out[64]: 0.6165784385797879
```

```
In [65]: ▶ cat_pred1 = cat.predict(X_test)  
rmse(cat_pred1, y_test)
```

```
Out[65]: 280.3755042555822
```

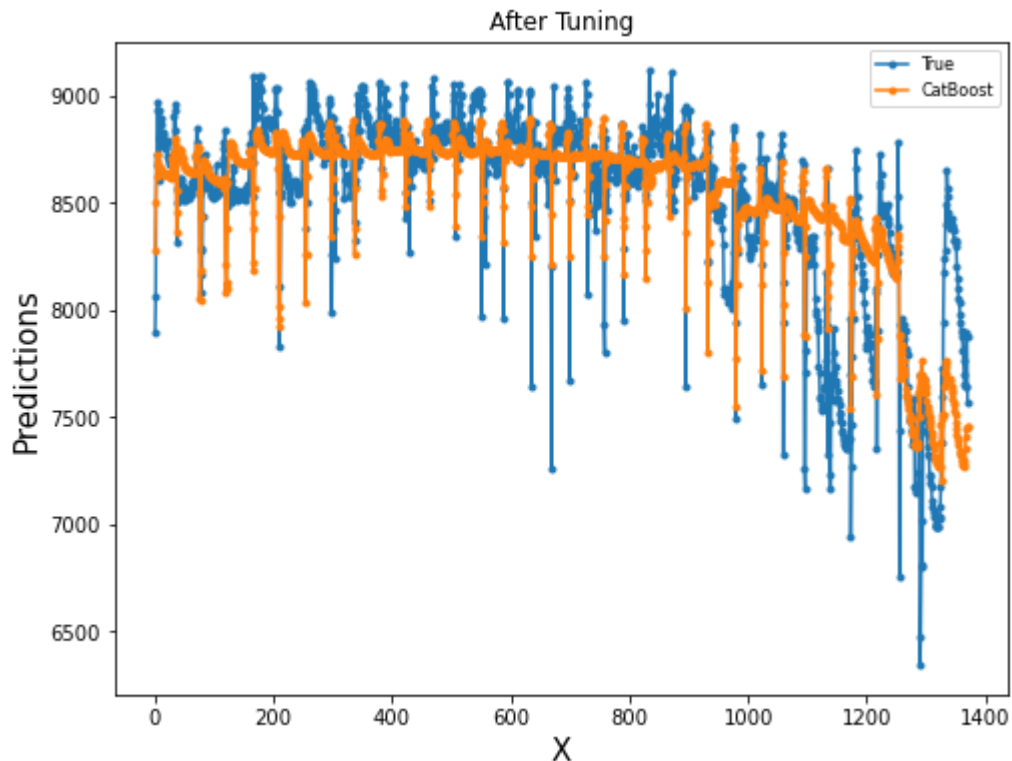
```
In [66]: ▶ # x1 = [i for i in range(4)]  
x = [i for i in range(len(cat_pred1))]
```

```
In [67]: ▶ plt.figure(figsize=(8, 6))
plt.plot(x, y_test, marker='.', label="True")
plt.plot(x, rf_pred, marker='.', label="Random Forest")
plt.plot(x, xg_pred, marker='.', label="XGBoost")
plt.plot(x, cat_pred, marker='.', label="CatBoost")
#plt.plot(x, inv_yhat, marker='.', label="LSTM")
plt.ylabel('Predictions', size=15)
plt.xlabel('X', size=15)
plt.title("Before Tuning")
plt.legend(fontsize=8)
plt.show()
```





```
In [68]: ▶ plt.figure(figsize=(8, 6))
plt.plot(x, y_test, marker='.', label="True")
plt.plot(x, cat_pred1, marker='.', label="CatBoost")
plt.ylabel('Predictions', size=15)
plt.xlabel('X', size=15)
plt.title("After Tuning")
plt.legend(fontsize=8)
plt.show()
```



CatBoost shows a significant improvement in the root mean square error after hyperparameter tuning which is an indication of more precise prediction of out of sample data

## Suggestion/Limitation

The Time variable was encoded which did not allow effective decomposition of time

We started with observing the heatmap of our data and `perf_max_depth_MD`, `perf_shot_bottom_depth_MD` and `perf_shot_top_depth` to be the top 3 positively correlated features.

Feature Importance technique using `RandomForestRegression` stated that `Discharge_rate_ADC1`, `Perc_stage`, `well_stage`, `Prop_rate`, `Inline_prop conc well_name`, `Perf_max_deph_MD` are the most important. XGBoost gave us good results overall.

In [ ]: ▶